

Đã xử lý trong 50s >

Có 2 cuộc migration độc lập, không nên gộp chung:

1. JDK 8 → JDK 21: thay đổi phiên bản Java/bytecode.
2. Java EE `javax.*` → Jakarta EE `jakarta.*` : thay đổi tên package/API.

Vì vậy, “chạy JDK 21” không đồng nghĩa mọi `javax.*` đều phải đổi thành `jakarta.*`.

1. Module A chạy JDK 21 dùng JAR của module B build bằng Java 8

Về bytecode, thông thường không có vấn đề:

- Module B build bằng Java 8 tạo class file version 52.
- JVM 21 có thể chạy class file được build bởi Java 8.
- Chiều ngược lại mới không được: JVM 8 không chạy được JAR build bằng Java 21.

Vì vậy:

```
Module B: Java 8
    ↓ JAR
Module A: Java 21
```



Hoàn toàn có thể build và chạy, miễn là B không phụ thuộc vào API đã bị xóa hoặc thay đổi trong JDK mới.

Oracle cũng cho biết code sử dụng các Java SE API chính thức nhìn chung tiếp tục hoạt động, nhưng một số module Java EE như JAXB, JAX-WS và Common Annotations đã bị loại khỏi JDK kể từ JDK 11. Khi đó phải khai báo dependency riêng. Oracle JDK 21 Migration Guide ↗, Removed Java EE modules ↗.

Không phải mọi `javax.*` đều đổi thành `jakarta.*`

Một số package `javax` vẫn thuộc Java SE và vẫn có trong JDK 21, ví dụ:

```
javax.crypto.*
javax.net.*
javax.sql.*
javax.management.*
javax.xml.parsers.*
```

Những package này không cần đổi sang Jakarta.

Các package Java EE thường phải migrate gồm:

javax.servlet	→	jakarta.servlet	
javax.persistence	→	jakarta.persistence	
javax.validation	→	jakarta.validation	
javax.annotation	→	jakarta.annotation	
javax.transaction	→	jakarta.transaction	
javax.ws.rs	→	jakarta.ws.rs	
javax.xml.bind	→	jakarta.xml.bind	

2. Khi nào module A build được?

Giả sử module B có API:

```
public class LegacyService {
    public void process(javax.servlet.http.HttpServlet request) {
    }
}
```

Trong module A bạn có:

```
jakarta.servlet.http.HttpServletRequest request;
legacyService.process(request);
```

Module A sẽ không compile vì:

```
jakarta.servlet.http.HttpServletRequest 
```

và:

```
javax.servlet.http.HttpServletRequest 
```

là hai kiểu dữ liệu hoàn toàn khác nhau.

Jakarta EE xác nhận việc chuyển namespace này không tương thích source và cũng không tương thích binary với các phiên bản Java EE trước.

[Jakarta EE specification 7](#).

Trường hợp build vẫn thành công

Ví dụ code của B:

```
public class LegacyService {  
    public void process() {  
        javax.persistence.EntityManager em = .  
    }  
}
```

A chỉ gọi:

```
legacyService.process();
```

Compiler của A có thể không cần phân tích đầy đủ phần implementation bên trong B, nên module A có khả năng vẫn compile và package thành công.

Nhưng điều đó không chứng minh ứng dụng sẽ chạy được.

3. Exclude javax và thêm jakarta có ép B dùng Jakarta được không?

Không.

Trước hết, lệnh:

```
mvn dependency:tree
```

chỉ hiển thị cây dependency, không thay đổi dependency. Việc loại dependency thực tế phải cấu hình <exclusions> trong pom.xml.

Ví dụ:

```
<dependency>  
    <groupId>com.example</groupId>  
    <artifactId>module-b</artifactId>  
    <version>1.0.0</version>  
  
    <exclusions>  
        <exclusion>  
            <groupId>javax.servlet</groupId>  
            <artifactId>javax.servlet-api</art
```

```
</exclusion>
</exclusions>
</dependency>

<dependency>
  <groupId>jakarta.servlet</groupId>
  <artifactId>jakarta.servlet-api</artifactId>
</dependency>
```

Cấu hình này chỉ làm:

Loại javax.servlet-api khỏi classpath
Thêm jakarta.servlet-api vào classpath



Nó không thay đổi bytecode bên trong module-
b.jar .

Nếu B đã compile với:

```
javax.servlet.http.HttpServletRequest
```

thì trong bytecode của B vẫn ghi chính xác tên:

```
javax/servlet/http/HttpServletRequest
```



JVM sẽ tìm đúng class đó. Nó không tự động
chuyển sang:

```
jakarta/servlet/http/HttpServletRequest
```



Maven exclusion chỉ loại transitive dependency
khỏi classpath, không sửa code hoặc bytecode
của dependency. Apache Maven dependency
exclusions ↗.

4. Build và runtime sẽ xảy ra chuyện gì?

Trường hợp	Compile/build A	Runtime
B chỉ dùng javax thuộc Java SE như javax.sql	Thường thành công	Thường hoặ

Trường hợp	Compile/build A	Runtime
B dùng Java EE javax , và dependency javax vẫn còn đầy đủ	Có thể thành công	Có thể chạy chưa chắc bị lỗi được với fra Jakarta
Exclude javax , thêm jakarta , trong khi public API của B có javax	Thường compile lỗi	Không chạy
Exclude javax , nhưng javax chỉ nằm trong implementation của B	Có thể vẫn build thành công	Lỗi khi code load hoặc tr
Giữ đồng thời javax và jakarta	Thường compile được	Hai hệ API t không thể tr object qua l
B đã được migrate/rebuild sang Jakarta	Thành công	Đây là phươ

Các lỗi runtime phổ biến:

```
java.lang.NoClassDefFoundError: javax/se
java.lang.ClassNotFoundException: javax.
java.lang.NoSuchMethodError
java.lang.LinkageError
java.lang.TypeNotPresentException
```

Ứng dụng cũng có thể fail ngay khi startup nếu Spring/Hibernate scan class, annotation hoặc entity của B.

Ví dụ, module B có:

```
@javax.persistence.Entity
public class Customer {
```

}

Trong khi A dùng Spring Boot 3 và Hibernate 6 theo Jakarta:

```
@jakarta.persistence.Entity
```

Hibernate Jakarta có thể không nhận Customer của B là một entity. Kết quả có thể là:

- Entity không được quản lý.
- Repository khởi tạo thất bại.
- Mapping lỗi khi startup.
- Not a managed type .

Tương tự:

- javax.servlet.Filter không phải jakarta.servlet.Filter .
- javax.validation.Valid không phải jakarta.validation.Valid .
- javax.annotation.PostConstruct có thể không được framework Jakarta xử lý như jakarta.annotation.PostConstruct .

5. Phương án xử lý nên dùng

Tốt nhất: migrate module B

Nếu có source code của B:

```
javax.servlet      → jakarta.servlet
javax.persistence → jakarta.persistence
javax.validation   → jakarta.validation
...
```



Sau đó:

- Nâng dependency của B sang phiên bản Jakarta.
- Build lại B trên JDK 17 hoặc 21.
- Chạy unit test và integration test.
- Publish một version mới, ví dụ module-b:2.0.0 .

Nếu B là thư viện bên thứ ba

Tìm phiên bản mới của B hỗ trợ:

- Jakarta EE 9/10 trở lên.
- Spring Framework 6.
- Spring Boot 3.
- JDK 17/21.

Nếu không có source và chưa có phiên bản Jakarta

Có thể cân nhắc Eclipse Transformer để biến đổi package reference trong JAR/WAR từ `javax` sang `jakarta`. Công cụ này có khả năng sửa class file và các resource liên quan. [Eclipse Transformer](#).

Tuy nhiên vẫn cần test kỹ vì ngoài class bytecode còn có thể có:

- XML descriptor.
- Reflection với tên class dạng chuỗi.
- `ServiceLoader`.
- Serialization.
- Framework metadata.
- File cấu hình chứa `javax.*`.

Giải pháp tạm thời

Giữ cả dependency `javax` và `jakarta` chỉ phù hợp khi B tương đối độc lập và không trao đổi các object `javax` với A.

Ví dụ interface giữa A và B chỉ dùng:

```
String  
int  
BigDecimal  
LocalDate  
DTO tự định nghĩa
```

Không nên để boundary chứa:


```
javax.servlet.*  
javax.persistence.*  
javax.validation.*
```

Kết luận

- JAR của B build bằng Java 8 có thể được A chạy trên JDK 21.
- `javax` và `jakarta` là hai namespace khác nhau, không phải hai version có thể thay thế bằng Maven.
- Exclude `javax` rồi thêm `jakarta` không ép được B chuyển sang Jakarta.
- Build có thể vẫn thành công nếu A chưa chạm đến phần `javax` của B.
- Runtime vẫn có thể lỗi khi JVM hoặc framework load các class đó.
- Phương án an toàn nhất là nâng version hoặc rebuild module B sang Jakarta.
- Nếu B chỉ dùng các package Java SE như `javax.sql`, `javax.crypto` thì không cần chuyển đổi.



giải thích kỹ hơn về các ý trên
và các vấn đề có thể xảy ra
khi migration như đối với các
thư viện trên và cách làm trên

Đã xử lý trong 1m 10s >

Để hiểu đúng tình huống này, mình nên tách migration thành 4 lớp độc lập:

1. Tương thích bytecode giữa Java 8 và Java 21.
2. Thay đổi namespace `javax.*` → `jakarta.*`.

3. Tương thích giữa framework và implementation như Spring, Hibernate, Tomcat.
4. Dependency có mặt ở compile-time nhưng có thể không có ở runtime.

Một dự án có thể vượt qua bước `mvn clean package` nhưng vẫn chết ngay khi startup hoặc chỉ lỗi khi một API cụ thể được gọi.

1. Java 8 JAR chạy trên Java 21 có được không?

Thông thường là được.

Module B build bằng JDK 8 tạo bytecode có class-file version 52:

```
Module B source
  ↓ javac 8
module-b.jar – class version 52
  ↓
JVM 21 có thể đọc và chạy
```



JVM mới được thiết kế để chạy phần lớn bytecode được tạo bởi các JDK cũ. Oracle cũng nói code chỉ sử dụng Java SE API chính thức nhìn chung tiếp tục hoạt động, nhưng vẫn có các thay đổi về API, encapsulation và hành vi cần kiểm tra. [Oracle JDK 21 Migration Guide](#).

Chiều ngược lại thì không được:

```
Module A build bằng Java 21
→ class-file version 65
→ không chạy trên JVM 8
```



Khi đó sẽ có lỗi:

```
UnsupportedClassVersionError:
class file version 65.0,
this runtime only recognizes up to 52.0
```



Vì vậy, bản thân việc B được build bằng Java 8 không phải nguyên nhân chính. Vấn đề là B đã gọi những API nào và các API đó có tồn tại, có tương thích trên runtime mới hay không.

2. JDK 21 không đồng nghĩa phải bỏ tất cả javax

Đây là điểm rất dễ nhầm.

Có hai nhóm javax .

Nhóm 1: javax thuộc Java SE

Ví dụ:

```
javax.crypto.*  
javax.net.*  
javax.sql.*  
javax.management.*  
javax.xml.parsers.*  
javax.naming.*
```

Các package này vẫn là Java SE API. Chúng không được đổi thành jakarta.* .

Ví dụ module B dùng:

```
import javax.sql.DataSource;  
import javax.crypto.Cipher;
```

thì thông thường không cần migrate sang Jakarta. Module A chạy trên JDK 21 vẫn có thể sử dụng module B bình thường.

Nhóm 2: javax từng thuộc Java EE

Ví dụ:

```
javax.servlet.*
javax.persistence.*
javax.validation.*
javax.annotation.*
javax.transaction.*
javax.ws.rs.*
javax.xml.bind.*
javax.mail.*
```

Từ Jakarta EE 9, các API thuộc nhóm này chuyển sang namespace `jakarta.*`.

Ví dụ:

```
javax.servlet.Filter
→ jakarta.servlet.Filter
```



Đây không phải đổi tên mang tính hiển thị. Đối với JVM, chúng là hai class hoàn toàn khác nhau.

Jakarta EE 9 không tương thích source code và cũng không tương thích binary với Java EE cũ do việc đổi namespace. Jakarta EE Platform specification 7.

3. Tại sao javax và jakarta không thay thế trực tiếp cho nhau?

Giả sử module B được build bằng Java 8:

```
package com.example.legacy;

import javax.servlet.http.HttpServletRequest;

public class LegacyService {

    public void process(HttpServletRequest req) {
        // ...
    }
}
```

Sau khi compile, bytecode trong B lưu method descriptor tương đương:

```
process(  
    Ljavax/servlet/http/HttpServletRequest  
)V
```



Trong khi Jakarta Servlet định nghĩa:

```
Ljakarta/servlet/http/HttpServletRequest
```



Hai tên này không giống nhau.

Cho dù hai interface có gần như cùng method:

```
javax.servlet.http.HttpServletRequest  
jakarta.servlet.http.HttpServletRequest
```

thì JVM vẫn coi chúng là hai kiểu dữ liệu không liên quan.

Ví dụ module A:

```
import jakarta.servlet.http.HttpServletRequest  
  
public void handle(  
    HttpServletRequest request,  
    LegacyService legacyService  
) {  
    legacyService.process(request);  
}
```

Sẽ compile lỗi:

```
incompatible types:  
jakarta.servlet.http.HttpServletRequest  
cannot be converted to  
javax.servlet.http.HttpServletRequest
```



Không thể cast như sau:

```
<</> java
```

Vì object do Tomcat Jakarta tạo chỉ implement `jakarta.servlet.http.HttpServletRequest`, không implement interface `javax.servlet.http.HttpServletRequest`.

4. Maven exclude thật sự làm gì?

Giả sử dependency ban đầu:

```
module-a
├─ module-b
│   └─ javax.servlet-api
```

Bạn exclude `javax.servlet-api` và thêm Jakarta:

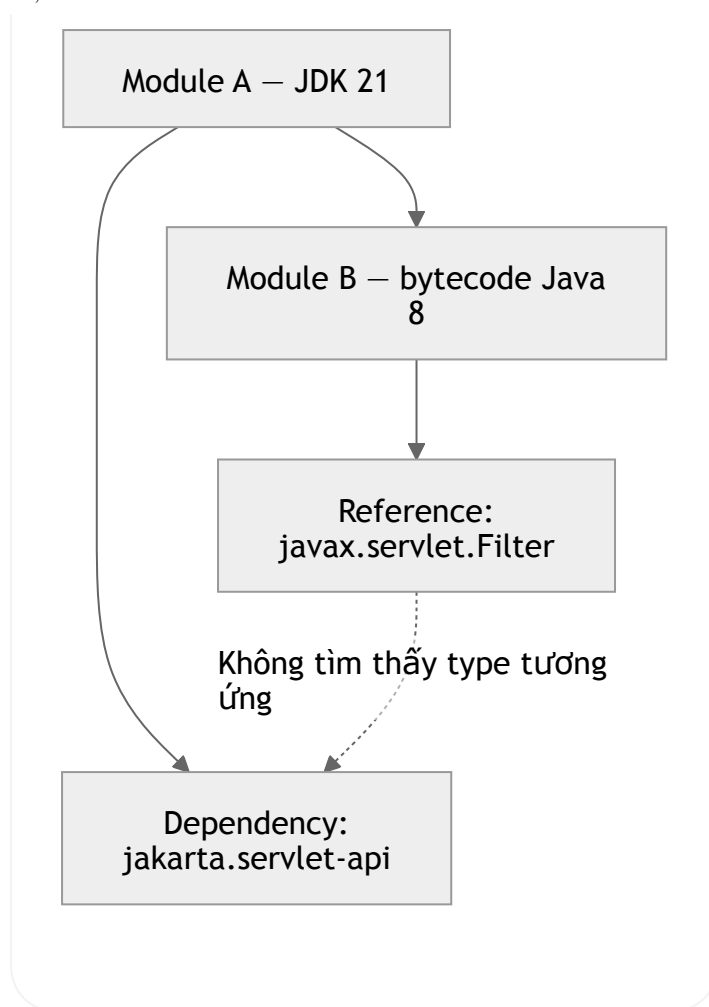
```
module-a
├─ module-b
│   └─ jakarta.servlet-api
```

Nhưng bên trong bytecode của `module-b.jar` vẫn là:

```
javax/servlet/Filter
javax/servlet/http/HttpServletRequest
```

Maven không sửa bytecode của B.

Mermaid </> </>



Maven exclusion chỉ ngăn một artifact transitive được thêm vào classpath. Nó không đổi import, method descriptor, annotation hoặc tên class bên trong JAR. Apache Maven dependency exclusions ↗.

Ngoài ra:

```
mvn dependency:tree
```

chỉ đọc và hiển thị dependency tree. Nó không exclude dependency.

Muốn exclude phải cấu hình trong `pom.xml` :

```
<dependency>
  <groupId>com.example</groupId>
  <artifactId>module-b</artifactId>
  <version>1.0.0</version>
```

```
<exclusions>
  <exclusion>
    <groupId>javax.servlet</groupId>
    <artifactId>javax.servlet-api</art
  </exclusion>
</exclusions>
</dependency>
```

Sau đó thêm Jakarta:

```
<dependency>
  <groupId>jakarta.servlet</groupId>
  <artifactId>jakarta.servlet-api</artifactId>
</dependency>
```

Cấu hình trên không "ép B sử dụng Jakarta". Nó chỉ tạo ra tình trạng:

B cần javax.servlet.Filter
classpath chỉ có jakarta.servlet.Filter



Vì vậy rất dễ phát sinh lỗi runtime.

5. Vì sao build có thể thành công nhưng runtime lại lỗi?

Compile module A không có nghĩa compiler kiểm tra toàn bộ implementation bên trong module B.

Giả sử B:

```
public class LegacyService {

    public void execute() {
        javax.xml.bind.JAXBContext context = .
    }
}
```

A chỉ gọi:


```
1 java.lang.NoClassDefFoundError:
```

```
2 java.lang.NoClassDefFoundError:
```

Public signature của `execute()` chỉ là:

```
void execute()
```

Nó không chứa `javax.xml.bind.JAXBContext`.

Vì vậy compiler có thể build A thành công.

Nhưng khi runtime chạy vào `execute()`, JVM

mới cần load `javax.xml.bind.JAXBContext`.

Nếu dependency đã bị exclude, nó sẽ lỗi:

```
java.lang.NoClassDefFoundError:
javax/xml/bind/JAXBContext
```



Thời điểm lỗi không hoàn toàn cố định. Nó có thể xuất hiện:

- Ngay khi application startup.
- Khi Spring scan bean.
- Khi Hibernate scan entity.
- Khi JVM load một class.
- Khi request đầu tiên đi vào API.
- Khi scheduled job chạy.
- Khi một nhánh ít dùng được thực thi.

Đó là lý do lỗi migration đôi khi "production mới gặp", dù compile và unit test đều pass.

6. Các loại lỗi có thể phát sinh

6.1 ClassNotFoundException

Một đoạn code chủ động yêu cầu class loader tìm class nhưng class không tồn tại:

```
Class.forName("javax.xml.bind.JAXBContext");
```

Lỗi:

ClassNotFoundException: javax.xml.bind.J 

Thường gặp với:

- Reflection.
- SPI/provider loading.
- Framework load class từ cấu hình.
- Tên class viết trong file properties/XML.

6.2 NoClassDefFoundError

JVM đang load B nhưng không tìm thấy class mà B đã được compile cùng:

NoClassDefFoundError: javax/servlet/Filt 

Ví dụ:

```
public class LegacyFilter implements javax.ser  
}
```

Chỉ cần Spring hoặc Tomcat load
LegacyFilter , JVM đã phải tìm
javax.servlet.Filter .

6.3 NoSuchMethodError

Có class, nhưng version runtime không có
method mà B đã dùng khi compile:

NoSuchMethodError:
some.library.Class.someMethod(...)



Thường do:

- B build với library version mới.
- Maven runtime lại chọn library version cũ.
- dependencyManagement ép một version không tương thích.
- Một dependency khác kéo vào cùng artifact nhưng version khác.

Đây là lỗi binary compatibility, không nhất thiết
liên quan trực tiếp đến javax .

6.4 AbstractMethodError

Class implementation cũ không implement method mới được thêm vào interface runtime.

Ví dụ:

```
AbstractMethodError:
Receiver class X does not define impleme
```



Thường gặp khi API và implementation không cùng generation.

6.5 IncompatibleClassChangeError

Một thứ từng là class nhưng runtime lại là interface, hoặc ngược lại; hoặc cấu trúc binary thay đổi không tương thích.

6.6 InaccessibleObjectException

Từ JDK 17 trở lên, strong encapsulation chặt hơn. Library cũ dùng reflection để truy cập field/private API của JDK có thể lỗi:

```
java.lang.reflect.InaccessibleObjectExce
```



Ví dụ library cũ truy cập nội bộ:

```
java.lang.ClassLoader
java.lang.reflect
sun.misc.*
com.sun.*
```



Oracle lưu ý JDK 17 trở lên không còn mặc định cho phép illegal reflective access như giai đoạn JDK 9–16. Oracle migration guide ↗.

Có thể dùng tạm:

```
--add-opens java.base/java.lang=ALL-UNNAMED
```

Nhưng đây chỉ nên là workaround ngắn hạn. Giải pháp đúng là nâng library.

6.7 Unsupported class file major version 65

Trường hợp thú vị là JVM 21 đọc được B Java 8, nhưng một library cũ trong B lại không đọc được class Java 21.

Ví dụ:

- ASM cũ.
- Byte Buddy cũ.
- CGLIB cũ.
- Mockito cũ.
- JaCoCo cũ.
- Plugin instrumentation/APM cũ.

Library đó tự phân tích bytecode của A và báo:

Unsupported class file major version 65 

Ở đây lỗi không phải JVM 21 không đọc được Java 8. Lỗi là bytecode parser cũ không hiểu class version 65 của Java 21.

6.8 Build plugin không tương thích JDK 21

Có thể lỗi trước cả khi chạy ứng dụng:

- Maven Compiler Plugin quá cũ.
- Maven Surefire/Failsafe quá cũ.
- Lombok cũ.
- MapStruct processor cũ.
- JaCoCo cũ.
- Checkstyle/SpotBugs/plugin dùng compiler internals.
- Maven hoặc Gradle version quá cũ.

Ví dụ:

IllegalAccessError
NoSuchFieldError: JCTree...
Unsupported class file major version 65 

Vì vậy migration JDK không chỉ nâng:

```
<java.version>21</java.version>
```

mà còn phải nâng toàn bộ build toolchain.

7. Các vấn đề cụ thể theo từng nhóm thư viện

Nhóm	Java EE cũ	Jakarta mới
Servlet	javax.servlet.*	jakarta.servlet.*
JPA	javax.persistence.*	jakarta.persistence.*
Validation	javax.validation.*	jakarta.validation.*
Annotation	javax.annotation.*	jakarta.annotation.*
Transaction	javax.transaction.*	jakarta.transaction.*
JAXB	javax.xml.bind.*	jakarta.xml.bind.*
JAX-RS	javax.ws.rs.*	jakarta.ws.rs.*

Nhóm	Java EE cũ	Jakarta mới
Mail	<code>javax.mail.*</code>	<code>jakarta.mail</code>
WebSocket	<code>javax.websocket.*</code>	<code>jakarta.websocket.*</code>

Servlet

Tomcat 9 sử dụng:

```
javax.servlet.*
```

Tomcat 10 trở lên sử dụng:

```
jakarta.servlet.*
```

Apache Tomcat xác nhận đây là breaking change và ứng dụng phải được compile lại với API mới. [Tomcat 10 Migration Guide ↗](#).

Ví dụ B:

```
public class AuthenticationFilter implements javax.servlet.Filter {  
}
```

A chạy Spring Boot 3 với Tomcat 10:

```
@Bean  
public FilterRegistrationBean<AuthenticationFilter> filterRegistrationBean() {  
    // ...  
}
```

Sẽ không compile vì generic yêu cầu:

```
jakarta.servlet.Filter
```

Nếu dùng reflection để né compile check thì runtime vẫn không đăng ký được vì `AuthenticationFilter` không phải Jakarta Filter.

JPA/Hibernate

Hệ cũ thường là:

```
Spring Boot 2  
Hibernate 5  
JPA 2.2  
javax.persistence
```



Hệ mới:

```
Spring Boot 3  
Hibernate 6  
Jakarta Persistence 3.x  
jakarta.persistence
```



Hibernate 6 yêu cầu ứng dụng đổi từ `javax.persistence.*` sang `jakarta.persistence.*`, đồng thời có nhiều thay đổi khác về query, type system, sequence và cấu hình. [Hibernate ORM 6 Migration Guide ↗](#).

Giả sử entity trong B:

```
@javax.persistence.Entity  
public class Customer {  
}
```

Hibernate 6 của A tìm annotation:

```
jakarta.persistence.Entity
```

Do đó `Customer` có thể không được coi là entity:

Not a managed type: class com.example.Cu 

Hoặc quan hệ từ entity Jakarta của A sang entity javax của B không được tạo mapping.

Validation

B:

```
public class Request {  
  
    @javax.validation.constraints.NotNull  
    private String name;  
}
```

A dùng Jakarta Validation:

```
jakarta.validation.Validator
```

Validation provider mới có thể không xem annotation cũ là constraint Jakarta. Kết quả nguy hiểm là ứng dụng không báo lỗi startup, nhưng dữ liệu không còn được validate như kỳ vọng.

Đây là loại lỗi silent behavior change, thường nguy hiểm hơn `ClassNotFoundException`.

JAXB

Trong Java 8, JAXB từng được cung cấp cùng JDK. Từ JDK 11, các Java EE module như JAXB và JAX-WS bị loại khỏi JDK. Vì vậy B có thể chạy trên Java 8 nhưng lỗi trên Java 21:

NoClassDefFoundError: javax/xml/bind/JAX 

Nếu B vẫn dùng `javax.xml.bind`, bạn phải:

- Giữ JAXB 2.x API và implementation tương thích `javax`; hoặc
- Migrate B sang JAXB 3/4 dùng `jakarta.xml.bind`.

Chỉ thêm `jakarta.xml.bind-api` không đáp ứng reference `javax.xml.bind.JAXBContext`.

8. Một bầy về tên artifact Jakarta

Không thể chỉ nhìn thấy groupId `jakarta.*` rồi kết luận package bên trong là `jakarta.*`.

Giai đoạn Jakarta EE 8, nhiều artifact đã đổi group/artifact name sang Jakarta nhưng package Java bên trong vẫn là `javax.*`.

Ví dụ theo các thể hệ phổ biến:

API	Version cũ còn package javax	Version đổi sang jakarta
Persistence	2.2	3.0+
Servlet	4.0	5.0+
Validation	2.0	3.0+
JAXB	2.3	3.0+
JAX-RS	2.1	3.0+
Annotation	1.3	2.0+
Transaction	1.3	2.0+
Mail	1.6	2.0+

Ví dụ artifact:

```
<groupId>jakarta.persistence</groupId>
<artifactId>jakarta.persistence-api</artifactId>
<version>2.2.3</version>
```

có thể vẫn cung cấp package:

```
javax.persistence.*
```

Trong khi version 3.x mới cung cấp:

```
jakarta.persistence.*
```

Do đó cần kiểm tra version và package thực tế, không chỉ dựa vào tên dependency.

9. Khi nào việc exclude dependency cũ là hợp lệ?

Exclude có thể hợp lệ trong các trường hợp sau.

Trường hợp 1: Cùng package và binary-compatible

Ví dụ B kéo:

```
slf4j-api 1.7.x
```



A sử dụng:

```
slf4j-api 1.7.y
```



Nếu hai version tương thích, có thể exclude version transitive và để Maven sử dụng version do A quản lý.

Nhưng vẫn phải kiểm tra major-version incompatibility.

Trường hợp 2: B không dùng feature đó

Ví dụ B khai báo một JDBC driver transitive nhưng chức năng bạn dùng không cần driver đó. Có thể exclude.

Trường hợp 3: Runtime/container đã cung cấp đúng API

Ví dụ WAR chạy trên application server và Servlet API được container cung cấp.
Dependency có thể để `provided`.

Tuy nhiên API container cung cấp phải đúng namespace:

```
Tomcat 9 → javax.servlet
Tomcat 10 → jakarta.servlet
```



Không hợp lệ

Không nên dùng:

```
exclude javax
+ add jakarta
```



với kỳ vọng Jakarta thay thế binary cho Javax.

10. Giữ đồng thời javax và jakarta có được không?

Về classpath, có thể tồn tại:

```
javax.servlet.Filter
jakarta.servlet.Filter
```



vì chúng có fully-qualified class name khác nhau.

Nhưng việc cùng tồn tại không có nghĩa chúng tương tác được.

Ví dụ:

```
Tomcat 10 tạo Jakarta HttpServletRequest
```



↓

```
B yêu cầu Javax HttpServletRequest
```

↓

```
Không truyền được object
```

Đối với JPA, có thể còn phức tạp hơn:

- Hibernate 5 quản lý entity javax .
- Hibernate 6 quản lý entity jakarta .
- Hai persistence context khác nhau.
- Hai transaction manager khác nhau.
- Lazy proxy không tương thích.
- Entity từ hệ này không tự động trở thành entity của hệ kia.

Giữ song song chỉ khả thi khi B được cô lập và API giao tiếp giữa A và B chỉ dùng Java SE hoặc DTO thuần:

```
String
long
BigDecimal
LocalDate
byte[]
CustomDto
```

Không để boundary A-B chứa:

```
HttpServletRequest
EntityManager
Validator
Transaction
ServletContext
```

Nếu B là hệ thống lớn không thể migrate, tách B thành service riêng thường an toàn hơn:

```
Module A – Jakarta/JDK 21
    ↓ REST/Kafka
Legacy service B – javax/JDK cũ
```



Boundary lúc này là JSON/event, không phải Java object.

11. Cách migration an toàn

Bước 1: Xác định stack hiện tại và stack đích

Ví dụ:

Thành phần	Hiện tại	Mục tiêu
Java	8	21
Spring Boot	2.x	3.x
Spring Framework	5.x	6.x
Hibernate	5.x	6.x
Servlet container	Tomcat 9	Tomcat 10.1
Persistence	javax.persistenc e	jakarta.pers: e
Validation	javax.validation	jakarta.valic

Nếu dùng Spring Boot 3, Spring hướng dẫn phải tránh các Java EE dependency cũ và sử dụng Jakarta API tương ứng. Spring Boot 3 dùng Spring Framework 6 và Jakarta EE 10 APIs. [Spring Boot 3 Migration Guide ↗](#).

Bước 2: Kiểm tra module B

Chạy:

```
jdeps --multi-release 21 \  
--recursive \  
--ignore-missing-deps \  
module-b.jar
```

Tìm dependency tới Javax:

```
jdeps --multi-release 21 \  
-verbose:classpath \  
module-b.jar
```

```
--ignore-missing-deps \  
module-b.jar | rg 'javax\.'
```

Kiểm tra việc sử dụng JDK internal APIs:

```
jdeps --jdk-internals module-b.jar
```

Xem public API của một class:

```
javap \  
-classpath module-b.jar \  
-public \  
com.example.LegacyService
```

Xem descriptor trong bytecode:

```
javap \  
-classpath module-b.jar \  
-s \  
com.example.LegacyService
```

Lưu ý: `jdeps` có thể không phát hiện đầy đủ:

- Tên class trong chuỗi.
- Reflection.
- XML/YAML/properties.
- Service provider.
- Dependency bị đóng gói lồng trong fat JAR.

Bước 3: Kiểm tra Maven dependency tree

```
mvn dependency:tree -Dverbose
```

Tìm Java EE cũ:

```
mvn dependency:tree -Dverbose \  

```

```
-Dincludes=javax.*:*
```

Tìm Jakarta:

```
mvn dependency:tree -Dverbose \
-Dincludes=jakarta.*:*
```

Nhưng không nên chỉ tìm groupId javax.* , vì một số artifact legacy có groupId khác:

```
org.hibernate
org.glassfish
org.jboss
com.sun
org.apache
```



Cũng nên xem effective POM:

```
mvn help:effective-pom
```

Điều này giúp phát hiện version bị parent/BOM hoặc dependencyManagement override.

Bước 4: Migrate module B nếu có source

Đổi import:

```
javax.persistence.Entity
→ jakarta.persistence.Entity
```

Nâng dependency tương ứng:

```
<dependency>
  <groupId>jakarta.persistence</groupId>
  <artifactId>jakarta.persistence-api</artifactId>
</dependency>
```

Không chỉ tìm trong file .java ; phải kiểm tra cả:

```
application.properties
application.yml
persistence.xml
orm.xml
web.xml
validation.xml
META-INF/services/*
JSP/taglib
reflection strings
```



Ví dụ property:

```
javax.persistence.schema-generation.database.schema-name
```

cần kiểm tra và đổi sang namespace Jakarta tương ứng:

```
jakarta.persistence.schema-generation.database.schema-name
```

Sau đó build B thành version mới:

```
module-b:1.x    → bản Javax
module-b:2.x    → bản Jakarta
```



Đây là cách rõ ràng hơn việc âm thầm thay nội dung của cùng một version.

Bước 5: Nếu không có source code của B

Ưu tiên theo thứ tự:

1. Tìm version mới của B hỗ trợ Jakarta.
2. Yêu cầu team sở hữu B publish bản Jakarta.
3. Dùng Eclipse Transformer hoặc Tomcat Migration Tool.
4. Cô lập B sau adapter/service riêng.
5. Chỉ giữ song song Javax/Jakarta như giải pháp tạm thời.

Eclipse Transformer có thể sửa package reference trong class/JAR/WAR và một số

resource. Eclipse Transformer π .

Nhưng sau transform vẫn phải test:

- Reflection.
- META-INF/services .
- XML descriptor.
- Annotation scanning.
- Serialization.
- Framework extension.
- JAR đã ký.
- Dependency bị shade vào trong JAR.
- API đã thay đổi behavior ngoài việc đổi package.

Transformer giải quyết tốt việc đổi tên, nhưng không bảo đảm tương thích nếu API/framework đã thay đổi method hoặc hành vi.

Bước 6: Build bằng JDK 21 thật sự

Kiểm tra:

```
java -version
mvn -version
```

Cả hai phải trở tới JDK 21.

POM:

```
<properties>
  <java.version>21</java.version>
  <maven.compiler.release>21</maven.compiler.release>
</properties>
```

release kiểm soát đồng thời language level, bytecode target và Java SE API được phép sử dụng.

Nếu module B vẫn cần phục vụ consumer Java 8 thì không nên build đề thành Java 21. Nên publish artifact/version riêng.

Bước 7: Không bỏ qua test runtime

Không chỉ chạy:

```
mvn clean package -DskipTests
```

Nên chạy:

```
mvn clean verify
```

Sau đó chạy artifact đã đóng gói bằng đúng image/runtime production:

```
java -jar target/module-a.jar
```

Cần test ít nhất:

- Spring context startup.
- API gọi vào module B.
- Filter/interceptor/security.
- Entity scan và repository.
- Transaction commit/rollback.
- Bean Validation.
- JAXB/XML processing.
- Scheduled job.
- Message listener.
- Serialization/deserialization.
- Shutdown hook.
- Deployment trên đúng Tomcat/application server.

Để xem class thực tế được load từ JAR nào:

```
java -Xlog:class+load=info -jar target/module-
```

Việc này hữu ích khi có cùng một library nhưng nhiều version khác nhau.

12. Phương án phù hợp nhất cho trường hợp của bạn

Nếu module A dùng JDK 21 + Spring Boot 3/Jakarta, còn B dùng Java 8 + Java EE javax , thì nên phân loại B:

B chỉ dùng javax Java SE

Ví dụ:

```
javax.sql  
javax.crypto  
javax.net
```



Có thể giữ nguyên B, nhưng vẫn kiểm tra JDK internal API và library version.

B dùng Java EE javax chỉ trong implementation

A có thể build thành công, nhưng runtime có nguy cơ lỗi. Tốt nhất migrate B; tạm thời có thể giữ Javax dependency đúng version nếu B hoạt động độc lập.

Public API của B chứa Java EE javax

Ví dụ:

```
public EntityManager getEntityManager();  
public void process(HttpServletRequest request  
public Validator getValidator();
```

Không nên dùng trực tiếp B trong A Jakarta.

Cần:

- Migrate và rebuild B; hoặc
- Tạo phiên bản Jakarta của B; hoặc
- Tách B sau boundary REST/event.

B có annotation javax để framework A scan

Ví dụ:

```
@javax.persistence.Entity  
@javax.validation.constraints.NotNull  
@javax.annotation.PostConstruct
```

Đây là trường hợp rất dễ build thành công nhưng sai hành vi runtime. Cần migrate annotation trong B, không nên chỉ thêm cả hai API.

Kết luận quan trọng nhất là:

JDK 21 có thể chạy JAR Java 8



nhưng:

Jakarta không phải bản drop-in replaceme



và:

Maven exclude/add dependency không rewri



Với module B có liên quan đến Servlet, JPA, Validation, Transaction hoặc JAXB, phương án đáng tin cậy nhất vẫn là tạo một phiên bản B được migrate và compile lại theo Jakarta, sau đó chạy integration test toàn bộ trên JDK 21.



module B dùng javax có
tương thích ngược với module
A dùng jakarta không?

Không. Nếu module B dùng các API Java EE `javax.*`, còn module A dùng `jakarta.*`, thì hai module không tương thích ngược trực tiếp với nhau.

Tuy nhiên cần phân biệt rõ B đang dùng loại `javax` nào.

1. Nếu B dùng javax thuộc Java SE: tương thích

Ví dụ:

```
javax.sql.DataSource
javax.crypto.Cipher
javax.net.ssl.SSLContext
javax.xml.parsers.DocumentBuilder
javax.naming.InitialContext
```

Những package này vẫn tồn tại trên JDK 21 và không chuyển sang Jakarta.

B build Java 8 + javax.sql



A chạy Java 21 + Jakarta



Thông thường vẫn hoạt động

2. Nếu B dùng Java EE javax : không tương thích trực tiếp

Ví dụ:

javax.servlet.*	≠ jakarta.servlet.	
javax.persistence.*	≠ jakarta.persiste	
javax.validation.*	≠ jakarta.validati	
javax.annotation.*	≠ jakarta.annotati	
javax.transaction.*	≠ jakarta.transact	
javax.ws.rs.*	≠ jakarta.ws.rs.*	
javax.xml.bind.*	≠ jakarta.xml.binc	

Ví dụ B khai báo:

```
public void process(javax.servlet.http.HttpServlet
}
```

Trong khi A có:

```
jakarta.servlet.http.HttpServletRequest request
```

A không thể gọi:

```
legacyService.process(request);
```

Compiler sẽ báo:

```
jakarta.servlet.http.HttpServletRequest
cannot be converted to
javax.servlet.http.HttpServletRequest
```

Mặc dù hai interface có chức năng gần giống nhau, JVM vẫn coi chúng là hai type hoàn toàn khác nhau.

Tùy cách B sử dụng javax , kết quả sẽ khác nhau

Cách B dùng javax	Build module A	Runtime
Chỉ dùng javax Java SE	Thành công	Thường hoạt động
javax xuất hiện trong public method của B	Có thể compile lỗi khi A gọi method	Không thể truy cập object Jakarta
javax chỉ nằm trong implementation của B	Có thể build thành công	Có thể lỗi khi chạy code đó
B có entity/filter/annotation javax để A scan	Có thể build thành công	Framework Jakarta có thể không nhận diện
Giữ đủ cả javax và Jakarta trong classpath	Có thể build và load	Hai hệ tồn tại không tự chuyển đổi

Có trường hợp nào B javax vẫn chạy trong A Jakarta không?

Có, nhưng chỉ khi B được cô lập.

Ví dụ B:

```
public class LegacyCalculator {

    public BigDecimal calculate(String customer) {
        // B dùng javax bên trong
    }
}
```

Boundary giữa A và B chỉ dùng Java SE:

```
String
BigDecimal
LocalDate
DTO thuần
```



Nếu bạn cung cấp đầy đủ Javax API và implementation mà B cần, B có thể chạy độc lập bên trong ứng dụng A.

Nhưng nếu boundary chứa type Java EE:

```
HttpServletRequest
EntityManager
Validator
Transaction
ServletContext
```

thì gần như không thể dùng trực tiếp giữa hai module.

Ví dụ thực tế với Servlet

B:

```
public class LegacyFilter implements javax.servlet.Filter {
}
```

A chạy Spring Boot 3/Tomcat 10, cần:

```
jakarta.servlet.Filter
```

Kết quả:

```
LegacyFilter legacyFilter = new LegacyFilter()  
jakarta.servlet.Filter filter = legacyFilter;
```

Nếu giữ cả hai dependency thì class B có thể được load, nhưng Tomcat 10 vẫn không đăng ký LegacyFilter vì nó không phải Jakarta Filter.

Ví dụ với JPA

B:

```
@javax.persistence.Entity  
public class Customer {  
}
```

A dùng Hibernate 6:

```
jakarta.persistence.Entity
```

A có thể compile thành công, nhưng Hibernate 6 có thể không coi Customer là Jakarta entity:

Not a managed type: class Customer



Exclude Javax rồi thêm Jakarta không tạo ra tương thích

```
<exclusion>  
  <groupId>javax.servlet</groupId>  
  <artifactId>javax.servlet-api</artifactId>  
</exclusion>
```


rồi thêm:

```
<dependency>
  <groupId>jakarta.servlet</groupId>
  <artifactId>jakarta.servlet-api</artifactId>
</dependency>
```

không biến B thành Jakarta.

Bytecode của B vẫn tìm:

javax/servlet/Filter



trong khi Jakarta JAR chỉ có:

jakarta/servlet/Filter



Runtime sẽ có nguy cơ:

NoClassDefFoundError: javax/servlet/Filter



Kết luận

B build Java 8 → có thể chạy trên JVM 21



nhưng:

B dùng Java EE javax → không tương thích



B chỉ có thể tiếp tục sử dụng khi:

- javax đó thuộc Java SE; hoặc
- B được cô lập và có đầy đủ javax dependencies riêng;
- Boundary giữa A và B không sử dụng type javax / jakarta ;
- Framework Jakarta của A không cần scan entity, filter hoặc annotation javax trong B.

Phương án tốt nhất vẫn là tạo phiên bản mới của B sử dụng jakarta.*. Nếu chưa thể migrate B,

hãy đặt một adapter với DTO thuần hoặc tách B thành service riêng.



ChatGPT có thể mắc lỗi. Hãy kiểm tra các thông tin quan trọng.